

Fuel Supply Intelligence & Resilience Platform

Problem Understanding and Solution Approach

Team GTA 7 · Ishmam Tahmid · Farhan Tahsin Khan · Mahmudul Hasan Sakib · Kazi Badrul Hasan

1 Problem understanding

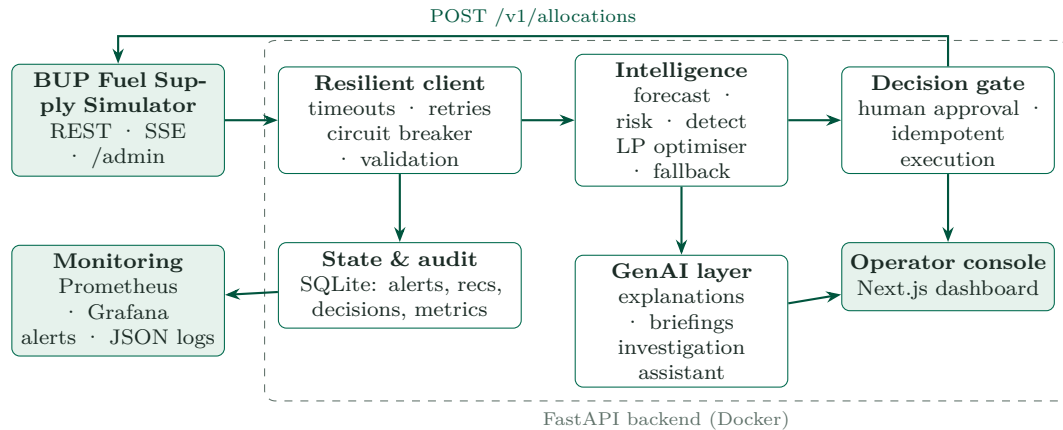
A fuel operations centre must keep four stations (Mirpur, Tongi, Karnaphuli, Cox's Bazar) supplied with Diesel, Petrol and Octane from two depots (Gazipur, Patiya) over six routes, while *demand*, *supply* and *infrastructure* change under it: demand spikes, delayed or reduced imports, constrained depots, disrupted routes and station outages. On top of the domain crises, the *software* itself fails: the simulator API can be slow, return errors, go down, serve stale data or drop its event stream.

Our reading of the brief: this is not a forecasting contest. The judges want a working, engineered operational system that closes the loop **Observe** → **Detect** → **Predict** → **Decide** → **Simulate** → **Act** → **Monitor** → **Recover**, keeps a human in charge of consequential decisions, and stays observable and usable when components fail (“*your job is to keep it working*”).

Constraints we design around

- **Hard physical limits** of the simulator on every shipment: route max shipment, depot inventory, depot dispatch capacity per tick (shared by all pending shipments), station capacity headroom, route/depot/station status, and a 1-tick departure plus transit delay.
- **Lead time:** fuel ordered now lands 3–5 ticks (45–75 min) later, so decisions must be made on *projected* inventory, not current inventory.
- **Scarcity:** during crises depot stock and dispatch capacity are shared across stations, so allocation is a constrained optimisation problem, not a per-station rule.
- **Unreliable integration:** REST is the source of truth, the SSE stream is only a hint, the database pool behind the simulator is small, and faults are injected deliberately.
- **Human review:** consequential simulated decisions must remain inspectable and approvable.

2 Solution architecture



Observe	Poll + SSE wake-up; full REST resync each tick; every response schema-validated (invalid → rejected + alert).
Detect	Anomalous demand (z-score vs forecast), abnormal inventory change, depot dispatch / low-stock bottlenecks, disruptions, delayed supply.
Predict	Per station × fuel demand forecast (hour-of-day profile × learned scale × live multiplier) with uncertainty band and rolling MAPE; stockout time and stockout probability over a 24 h window.
Decide	Linear-programming optimiser (SciPy HiGHS) with fair sharing under scarcity, 24 h cover target, backup cross-region routes, alternatives, confidence; validated against every simulator rule.
Simulate	What-if projection of inventory with vs without a proposed shipment (no writes).
Act	Operator approves/rejects (protected by operator key); <i>assisted</i> mode auto-executes only high-confidence, small shipments. Idempotency keys make retries safe.
Monitor	Prometheus metrics at three layers (application, system, intelligence), Grafana dashboard, alert rules, system-status panel.
Recover	Circuit breaker, degraded mode on cached state, automatic recovery and resync, rule-based fallback policy.

3 Intelligence approach

Principle: numbers from deterministic, constraint-aware code; language from the LLM. Quantities and routes are never produced by the LLM. The LLM (OpenAI → Gemini → Groq, with template fallback) turns structured facts into operator-facing explanations, incident summaries, situation briefings and answers to investigation questions grounded in current alerts and recommendations.

Every recommendation is *inspectable*, as §9 of the brief asks: current inventory, expected demand, projected stockout time, recommended shipment (quantity, depot, route, ETA), **expected impact** (stockout risk before → after, litres of unmet demand avoided), signals, binding constraints, confidence, and alternatives. Low confidence ⇒ human review requested.

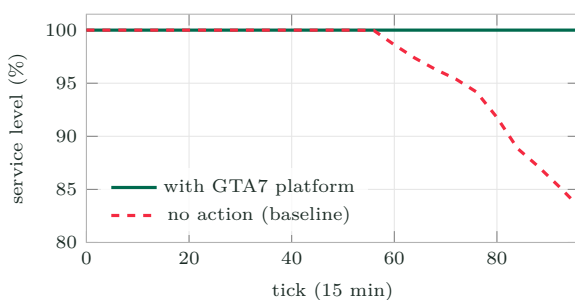
4 Resilience: defined behaviour for every failure

Failure	Platform behaviour (tested against injected faults)
Simulator unavailable (503)	Retries with backoff → degraded mode after 2 failed syncs (serves cached state, pauses auto-execution, critical alert) → breaker opens → half-open probe → automatic recovery + resync.
Error rate / high latency	Retries absorb transient errors; timeouts bound latency; hysteresis prevents alert flapping.
Stale data	Detected from the simulator header → warning alert; UI marks data as stale.
Stream disconnect	REST polling continues; SSE reconnects with backoff; full resync on reconnect.
Invalid response	Rejected by schema validation, last good state kept, integration alert raised.
Prediction/optimiser failure	Rule-based fallback allocation policy, fallback counted in metrics and alerted.
Low confidence	Recommendation flagged for human review; never auto-executed.
Rejected shipment (409)	Mapped to a clear reason (e.g. ROUTE_DISRUPTED); operator sees alternatives.

5 DevOps, observability and evidence

- **One command:** `docker compose up -build` starts simulator, backend, Prometheus and Grafana (web behind a profile), all with health checks.
- **CI/CD (GitHub Actions):** lint → 96 automated tests → web build → build images → deploy stack → health check → end-to-end smoke test (step the simulator, expect recommendations).
- **Observability:** request rate, p50/p95 latency, error rate, CPU/memory, simulator-call outcomes, breaker state, service level, open alerts, decisions per minute, prediction MAPE, model confidence, fallback activations; 8 Prometheus alert rules; structured JSON logs of decisions, integration failures and recoveries.
- **Reproducible scenarios:** 7 named crisis presets (incl. combined crisis) and 5 fault presets; decision audit export (CSV) and tick-by-tick replay.
- **Measured value:** A/B benchmark on the same deterministic world (same seed, same crisis script), *with* vs *without* the platform.

Measured result: combined crisis, 24 simulated hours, same seed



100%

service level
with platform

16,743 L

unmet demand
avoided

83.4%

without
platform

22 / 0

shipments sent /
rejected

Crisis script: Dhaka demand spike (×1.6, 8 h), Gazipur→Mirpur route disruption, supply delay, Patiya depot constrained. Same seed and events in both runs; the simulator is deterministic, so the difference is attributable to our decisions. Every shipment passed the simulator's validation (0 rejections).

6 Status and plan to the final demo

Component	Owner	Status
Backend core, integration, DevOps	Ishmam Tahmid	Working end to end: approve → shipment arrives; faults tested; CI green
Intelligence & GenAI Operator console	Farhan Tahsin Khan M. Hasan Sakib	Forecast, risk, detection, LP optimiser, LLM chain merged and tested Overview + network map, stations/depots, alerts, recommendation inbox with what-if/approve
Crisis, load test, evidence	Kazi Badrul Hasan	Crisis scenario scripts, fault tests, Grafana; k6 load test next

Next: connect the console to the live backend, k6 load test (avg/p50/p95/p99, throughput, error rate), resilience evidence document, and the 14-step demonstration story from §22 of the brief.